🧪 Flow-Wise Array Mastery — Practice Set

Design: This problem set is mapped 1:1 to the **Flow-Wise Array Mastery Curriculum**.
Instructions:

1. **Do not skip levels.** The curriculum builds complexity layer by layer.
2. **Timebox:** 20-30 minutes per problem.
3. **The Invariant Test:** If you cannot state the *invariant* (the rule that stays true inside your loop) in one sentence, you have not solved the problem, even if it passes the tests.

🟢 Level 1: Physical Layer (Basic Movement)

1A. Iteration Basics (Forward/Stride)

1. [LC 1920] Build Array from Permutation (Easy)

- **Maps to:** `1A.1 Forward Iteration`
- **Goal:** precise index mapping `ans[i] = nums[nums[i]]`.
- **Mental Check:** Did you confuse the index `i` with the value `nums[i]`?

2. [LC 1929] Concatenation of Array (Easy)

- **Maps to:** `1A.3 Stride/Modulo`
- **Goal:** Form an array of size `2n` using a single loop `0` to `2n-1`.
- **Technique:** Use `ans[i] = nums[i % n]`.

1B. Backward Iteration

3. [LC 88] Merge Sorted Array (Easy)

- **Maps to:** `1A.2 Backward Iteration`
- **Constraint:** You must update `nums1` in-place.
- **Invariant:** "Indices `p1`, `p2`, and `write` move backwards. `nums1[write]` is always the largest unused element."

1C. Simultaneous Iteration

4. [LC 1672] Richest Customer Wealth (Easy)

- **Maps to:** `1C Lockstep Iteration`
- **Goal:** Iterate through a matrix row-by-row.
- **Check:** Do you reset your `currentWealth` sum to 0 for every new row?

1D. Edge Cases

5. [LC 268] Missing Number (Easy)

- **Maps to:** `1D Edge Cases`
- **Goal:** Handle the case where the missing number is `n` (at the very end).
- **Technique:** Sum Formula `n*(n+1)/2` vs Iteration.

🔵 Level 2: Logical Layer (Index Arithmetic)

2A. Cyclic Arithmetic

6. [LC 189] Rotate Array (Medium)

- **Maps to:** `2A Cyclic Arithmetic`
- **Goal:** Rotate in-place.
- **Technique:** Use the "Three Reversals" trick (Reverse whole, Reverse 0 to k-1, Reverse k to end) OR cyclic replacements.

2B. 2D Mapping

7. [LC 74] Search a 2D Matrix (Medium)

- **Maps to:** `2B 2D-to-1D Mapping`
- **Goal:** Binary Search a matrix without copying it.
- **Formula:** `Row = Mid / Cols`, `Col = Mid % Cols`.

2C. Offset & Relative Indexing

8. [LC 118] Pascal's Triangle (Easy)

- **Maps to:** `2C Offset Indexing`
- **Goal:** `Val[i][j] = Val[i-1][j-1] + Val[i-1][j]`.
- **Check:** Did you guard the boundaries (j=0 and j=i)?

🟠 Level 3: Multi-View Layer (Dual Pointers)

3A. Reader/Writer (Compaction)

9. [LC 26] Remove Duplicates from Sorted Array (Easy)

- **Maps to:** `3A Reader/Writer`
- **Invariant:** `Nums[0...Write]` contains sorted unique elements.
- **Logic:** Only write when `Nums[Read] != Nums[Write-1]`.

10. [LC 283] Move Zeroes (Easy)

- **Maps to:** `3A Reader/Writer`
- **Invariant:** `Nums[0...Write-1]` are all non-zeroes.

11. [LC 27] Remove Element (Easy)

- **Maps to:** `3A Reader/Writer`

- **Goal:** Filter out a specific value `val` in-place.

3B. Converging Pointers (Pincer)

12. [LC 167] Two Sum II - Input Array Is Sorted (Medium)

- **Maps to:** `3B Converging Pointers`
- **Logic:** `Sum > Target` ? Decrement Right. `Sum < Target` ? Increment Left.
- **Why:** Sorted property allows eliminating search space.

13. [LC 11] Container With Most Water (Medium)

- **Maps to:** `3B Converging Pointers`
- **Invariant:** Moving the pointer at the **taller** height *never* yields a better result (width decreases, height is limited by the shorter side). Always move the shorter pointer.

14. [LC 125] Valid Palindrome (Easy)

- **Maps to:** `3B Converging Pointers`
- **Check:** Did you skip non-alphanumeric characters correctly?

3C. Diverging Pointers (Expansion)

15. [LC 5] Longest Palindromic Substring (Medium)

- **Maps to:** `3C Diverging Pointers`
- **Goal:** Expand from center `L--, R++` .
- **Critical:** Handle both `(i, i)` (odd length) and `(i, i+1)` (even length) centers.

3D. Fast & Slow Pointers

16. [LC 287] Find the Duplicate Number (Medium)

- **Maps to:** `3D Fast & Slow Pointers`
- **Concept:** Treat values as pointers (`index -> value` becomes `node -> next`).
- **Logic:** Floyd's Cycle Finding Algorithm.

🟣 Level 4: Range Layer (Subarrays & Windows)

4A. Fixed Sliding Window

17. [LC 643] Maximum Average Subarray I (Easy)

- **Maps to:** `4A Fixed Window`
- **Logic:** `Sum += New - Old` . Do not re-sum the whole window.

18. [LC 1343] Number of Sub-arrays of Size K and Average >= Threshold (Medium)

- **Maps to:** `4A Fixed Window`
- **Goal:** Count valid windows.

4B. Variable Sliding Window (Accordion)

19. [LC 209] Minimum Size Subarray Sum (Medium)

- **Maps to:** `4B Variable Window`
- **Goal:** Minimize window length.
- **Logic:** Expand Right until `Sum >= Target`. Then `while Sum >= Target`, update answer and shrink Left.

20. [LC 3] Longest Substring Without Repeating Characters (Medium)

- **Maps to:** `4B Variable Window`
- **State:** Hash Set or Boolean Array for characters in current window.
- **Logic:** If `char` exists in window, shrink Left until it's removed.

21. [LC 1004] Max Consecutive Ones III (Medium)

- **Maps to:** `4B Variable Window`
- **State:** Count of `Zeroes` in window.
- **Logic:** Shrink Left if `Zeroes > K`.

4C. Prefix Sums (1D)

22. [LC 303] Range Sum Query - Immutable (Easy)

- **Maps to:** `4C Prefix Sums`
- **Goal:** O(1) Range Sum.
- **Check:** Did you make your prefix array size `N+1`?

23. [LC 724] Find Pivot Index (Easy)

- **Maps to:** `4C Prefix Sums`
- **Logic:** `LeftSum == TotalSum - LeftSum - CurrentVal`.

4D. 2D Prefix Sums

24. [LC 304] Range Sum Query 2D - Immutable (Medium)

- **Maps to:** `4D 2D Prefix Sums`
- **Formula:** `Sum = P[r2][c2] - P[r1-1][c2] - P[r2][c1-1] + P[r1-1][c1-1]`.

4E. Difference Arrays

25. [LC 1109] Corporate Flight Bookings (Medium)

- **Maps to:** `4E Difference Arrays`
- **Goal:** Apply range updates O(1).
- **Logic:** `Diff[Start] += Seats`, `Diff[End+1] -= Seats`.

4F. Prefix Sum + HashMap

26. [LC 560] Subarray Sum Equals K (Medium)

- Maps to: `4F Prefix Sum + HashMap`
- Why: Sliding window fails on negative numbers.
- Logic: `Count += Map[CurrentPrefix - K]` .

27. [LC 523] Continuous Subarray Sum (Medium)

- Maps to: `4F Prefix Sum + HashMap`
- Logic: Store `Prefix % K` in map.

4G. Monotonic Deque

28. [LC 239] Sliding Window Maximum (Hard)

- Maps to: `4G Monotonic Deque`
- Invariant: Deque stores Indices. Values corresponding to indices are strictly Descending.

🔴 Level 5: Abstract Layer (Search & Partition)

5A/5B. Partitioning

29. [LC 75] Sort Colors (Medium)

- Maps to: `5B 3-Way Partitioning (DNF)`
- Goal: Sort 0, 1, 2 in one pass.
- Invariant: `[0...Low-1]` are 0s. `[High+1...End]` are 2s.

30. [LC 2161] Partition Array According to Given Pivot (Medium)

- Maps to: `5A 2-Way Partitioning` (Stable)
- Goal: Keep relative order. (Requires 3 passes or extra space usually, unlike DNF).

5C/5D. Binary Search Bounds

31. [LC 704] Binary Search (Easy)

- Maps to: `5C Classic Binary Search`
- Check: `Left <= Right` loop condition.

32. [LC 35] Search Insert Position (Easy)

- Maps to: `5D Lower Bound`
- Goal: Find the first index `>= Target` .

33. [LC 34] Find First and Last Position of Element (Medium)

- Maps to: `5D Lower/Upper Bound`
- Goal: Run BS twice.

5E. Binary Search on Answer Space

34. [LC 1011] Capacity To Ship Packages Within D Days (Medium)
    - Maps to: `5E BS on Answer`
    - Search Space: `[Max(Weights), Sum(Weights)]`.
    - Check: `IsFeasible(Capacity)` function uses a simple greedy loop.
35. [LC 875] Koko Eating Bananas (Medium)
    - Maps to: `5E BS on Answer`
    - Search Space: `[1, Max(Piles)]`.